บทที่ 5

การแสดงผลรายการข้อมูลและประสิทธิภาพการทำงาน

ความเร็วและความลื่นไหลในการเลื่อนดูข้อมูลยาว ๆ (Scrolling Performance) ถือเป็นปัจจัยหลักที่ส่งผลต่อคะแนนความพึงพอใจของผู้ใช้งานโมบายแอปพลิเคชัน ในบทเรียนนี้เราจะศึกษาข้อจำกัดของคอมโพเนนต์ `ScrollView` และเจาะลึกวิธีการนำเสนอรายการข้อมูลประสิทธิภาพสูงด้วย `FlatList` และ `SectionList` รวมถึงแนวทางการเชื่อมต่อข้อมูลสดผ่านเน็ตเวิร์ก

5.1 ปัญหาประสิทธิภาพของ ScrollView กับข้อมูลขนาดใหญ่

คอมโพเนนต์ `ScrollView` มีลักษณะการเรนเดอร์ข้อมูลแบบ **กระตือรือร้น (Eager Rendering)** หมายความว่า ทันทีที่คอมโพเนนต์ถูกเปิดขึ้นมารันบนหน้าจอ มันจะประมวลผลวาดโครงสร้างของคอมโพเนนต์ลูกย่อยทุกชิ้นภายในลิสต์ลงสู่หน่วยความจำ (RAM) พร้อมกันในคราวเดียว:

- **จุดเด่น:** ใช้งานง่าย รันคอมโพเนนต์ลูกที่แตกต่างรูปแบบกันได้อย่างอิสระ
- **จุดด้อยที่รุนแรง:** หากข้อมูลมีจำนวนมาก (เช่น เกิน 100 รายการขึ้นไป หรือมีรูปภาพความละเอียดสูงประกอบ) ระบบปฏิบัติการจะต้องสร้าง Native Views คาไว้ใน RAM มหาศาล ส่งผลให้แอปพลิเคชันเกิดการสะดุด เฟรมเรตตก (Frame Drops) และแอปเด้งดับเนื่องจากหน่วยความจำเต็ม (Out of Memory)

ดังนั้น `ScrollView` จึงควรเลือกใช้เฉพาะหน้ารายละเอียดทั่วไป หรือแบบฟอร์มที่มีจำนวนวัตถุคงที่สั้น ๆ เท่านั้น

5.2 การแก้ไขด้วย FlatList: Virtualization และ Recycling

เพื่อจัดการปัญหาข้อมูลล้นจอ React Native ได้จัดเตรียมคอมโพเนนต์ **`FlatList`** (React Native, 2024) (ซึ่งสืบทอดกลไกมาจาก `VirtualizedList`) โดยใช้เทคนิคสถาปัตยกรรมระดับเนทีฟในการจัดระเบียบพื้นที่หน่วยความจำดังนี้:

1. ระบบจำลองเสมือนจริง (Virtualization)

`FlatList` จะวาดเรนเดอร์เฉพาะคอมโพเนนต์รายการที่ **ปรากฏตัวอยู่ในขอบเขตหน้าจอจริงเท่านั้น** (บวกกับพื้นที่ข้างเคียงล่วงหน้าเล็กน้อย เรียกว่า Window Buffer) ข้อมูลที่อยู่นอกจอจะยังไม่ถูกนำมาประมวลผล

2. การหมุนเวียนมุมมองกลับมาใช้ใหม่ (Item View Recycling)

เมื่อผู้ใช้งานใช้นิ้วปัดหน้าจอเพื่อเลื่อนลง กล่อง View รายการที่เคลื่อนล้นหลุดขอบจอด้านบนออกไป จะถูกส่งกลับเข้าไปในคลังสะสมและนำโครงสร้างกล่องนั้นมา **"รีไซเคิลปรับเปลี่ยนข้อมูลข้างในใหม่"** เพื่อนำมารอแสดงผลเป็นรายการใหม่ที่วิ่งเข้ามาจากขอบจอด้านล่าง ส่งผลให้หน่วยความจำถูกจำกัดอยู่คงที่ตามขนาดจอเสมอ ไม่ว่าจะโหลดข้อมูล 100 หรือ 100,000 ชิ้นก็ตาม

5.3 โครงสร้างและพร็อพส์หลักของ FlatList (data, renderItem, keyExtractor)

การทำงานร่วมกับ `FlatList` จำเป็นต้องส่งค่าพร็อพส์ที่ระบุหน้าที่เฉพาะเจาะจง 3 ตัวหลักเสมอ:

• `data`: อาร์เรย์ของข้อมูลดิบที่ต้องการแสดงผล (เช่น `Array<any>`)

• `renderItem`: ฟังก์ชันคืนค่า JSX คอมโพเนนต์ที่อธิบายหน้าตาของแต่ละรายการย่อย โดยรับพารามิเตอร์ออบเจกต์ที่มีพร็อพส์ `{ item, index }`

• `keyExtractor`: ฟังก์ชันส่งสตริงคีย์ที่เป็นเอกลักษณ์ (Unique Key) ของรายการย่อยแต่ละชิ้น เพื่อช่วยให้ระบบ Diffing Engine ของ React ติดตามและระบุการเปลี่ยนแปลงตำแหน่งชิ้นงานได้ถูกต้องโดยไม่ต้องวาดใหม่ทั้งหมด

```
import { FlatList, Text, View, StyleSheet } from 'react-native';

const mockUsers = [
  { id: '1', name: 'สมชาย' },
  { id: '2', name: 'สมศรี' },
  { id: '3', name: 'สมหวัง' },
];

export default function UserList() {
  return (
    <FlatList
      data={mockUsers}
      keyExtractor={(item) => item.id} // ช่วยส่งคืนสตริงระบุคีย์ที่ไม่ซ้ำกัน
      renderItem={({ item }) => (
        <View style={styles.item}>
          <Text style={styles.itemText}>{item.name}</Text>
        </View>
   )}
  />
 );
}

const styles = StyleSheet.create({
  item: {
    padding: 16,
    borderBottomWidth: 1,
```

```
    borderBottomColor: '#ccc',
  },
  itemText: {
    fontSize: 16,
  },
});
```

5.4 การดึงข้อมูลจาก REST API มาใช้ร่วมกับ FlatList

ในระบบงานจริง ข้อมูลรายการมักจะดึงผ่านเน็ตเวิร์กจากเซิร์ฟเวอร์ภายนอก เราสามารถประยุกต์ใช้คำสั่ง fetch() ร่วมกับสถานะ useState และ useEffect ดึงข้อมูลมาอัปโหลดใส่ใน FlatList ดังตัวอย่างนี้:

```
import { useState, useEffect } from 'react';
import {
  FlatList,
  Text,
  View,
  ActivityIndicator,
  StyleSheet,
  SafeAreaView
} from 'react-native';

interface Post {
  id: number;
  title: string;
  body: string;
}

export default function ApiListScreen() {
  const [posts, setPosts] = useState<Post[]>([]);
  const [loading, setLoading] = useState<boolean>(true);

 // 1. ฟังก์ชันดึงข้อมูลจาก REST API ของ JSONPlaceholder (JSONPlaceholder, 2024)
  const fetchPosts = async () => {
    try {
     setLoading(true);
      const response = await
fetch('https://jsonplaceholder.typicode.com/posts?_limit=15');
      const data = await response.json();
      setPosts(data);
    } catch (error) {
      console.error('ดึงข้อมูลผิดพลาด:', error);
    } finally {
      setLoading(false);
    }
  };

  useEffect(() => {
```

```
    fetchPosts();
  }, []);

  // 2. แสดงตัวหมุนโหลดข้อมูลชั่วคราวหากเน็ตเวิร์กกำลังดึงข้อมูลอยู่
  if (loading) {
    return (
      <View style={styles.centerContainer}>
        <ActivityIndicator size="large" color="#38bdf8" />
        <Text style={styles.loadingText}>กำลังดึงข้อมูลโพสต์ล่าสุด...</Text>
      </View>
  );
  }

  return (
    <SafeAreaView style={styles.container}>
      <FlatList
        data={posts}
        keyExtractor={(item) => item.id.toString()}
        renderItem={({ item }) => (
          <View style={styles.postCard}>
            <Text style={styles.postTitle}>📌 {item.title}</Text>
            <Text style={styles.postBody}>{item.body}</Text>
          </View>
    )}
   />
    </SafeAreaView>
 );
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: '#0f172a',
  },
  centerContainer: {
    flex: 1,
    backgroundColor: '#0f172a',
    justifyContent: 'center',
    alignItems: 'center',
  },
  loadingText: {
    color: '#94a3b8',
    marginTop: 10,
  },
  postCard: {
    backgroundColor: '#1e293b',
    padding: 16,
    marginVertical: 8,
    marginHorizontal: 16,
```

```
    borderRadius: 12,
    borderWidth: 1,
    borderColor: '#334155',
  },
  postTitle: {
    fontSize: 16,
    fontWeight: 'bold',
    color: '#ffffff',
    textTransform: 'capitalize',
    marginBottom: 6,
  },
  postBody: {
    fontSize: 14,
    color: '#cbd5e1',
    lineHeight: 20,
  },
});
```

5.5 การปรับแต่งส่วนประกอบลิสต์ (Header, Footer, Item Separator, และ ListEmptyComponent)

`FlatList` เตรียมอินเทอร์เฟซอรรถประโยชน์อื่น ๆ ที่ทำให้การออกแบบหน้าตารายการทำได้สมบูรณ์แบบโดยไม่ต้องครอบคอมโพเนนต์ด้วย ScrollView ซ้ำซ้อน:

• **`ListHeaderComponent`**: พื้นที่จัดแสดงสำหรับวางหัวตารางหรือแผงควบคุมส่วนบน (เช่น ช่องแถบค้นหา โฆษณา หรือป้ายหัวข้อใหญ่) ซึ่งจะเลื่อนสไลด์ขึ้นไปตามความเร็วของลิสต์

• **`ListFooterComponent`**: พื้นที่จัดแสดงสำหรับวางท้ายตารางรายการ (เช่น ปุ่มแสดงหน้าถัดไป หรือตัวหมุนบอกว่ากำลังดาวน์โหลดข้อมูลต่อท้าย)

• **`ItemSeparatorComponent`**: คอมโพเนนต์คั่นกลางระหว่างชิ้นรายการเพื่อระบุช่องไฟหรือวาดเส้นแบ่งเส้นคั่น (จะตัดขอบไม่วาดท้ายรายการล่างสุดให้เอง)

• **`ListEmptyComponent`**: คอมโพเนนต์แสดงผลทดแทนเมื่ออาร์เรย์ `data` มีขนาดเป็นศูนย์ (ไม่มีข้อมูลใด ๆ ในรายการ) เพื่อแจ้งเตือนผู้ใช้อย่างชัดเจน

5.6 ฟีเจอร์ Pull-to-Refresh และ Infinite Scroll (onEndReached)

1. ระบบ Pull-to-Refresh (ดึงหน้าจอเพื่อรีเฟรชข้อมูล)

เป็นการเปิดโอกาสให้ผู้ใช้งานกวาดนิ้วลงเพื่ออัปโหลดใหม่เพื่อเอาโพสต์ชุดใหม่ โดยควบคุมด้วย 2 พร็อพส์หลัก:

• `refreshing`: ตัวแปรสถานะ Boolean ที่บ่งบอกว่าขณะนี้ระบบกำลังดาวน์โหลดรอบใหม่ค้างอยู่หรือไม่ (เพื่อให้ไอคอนหมุนทำงาน)

• `onRefresh`: ฟังก์ชันที่จะถูกสั่งงานอัตโนมัติเมื่อตรวจพบการลากนิ้วดึงหน้าจอลำดับล่างลงมา

2. ระบบ Infinite Scroll (เลื่อนลงสุดเพื่อดาวน์โหลดต่ออัตโนมัติ)

เทคนิคยอดนิยมบนหน้า Feed ของโซเชียลมีเดียเพื่อรองรับการโหลดข้อมูลแบบแบ่งหน้า (Pagination) ควบคุมผ่าน:

• `onEndReached`: ฟังก์ชันสั่งงานเมื่อผู้ใช้งานเลื่อนหน้าจอลิสต์เข้าใกล้ความยาวล่างสุดของรายการ

• `onEndReachedThreshold`: ค่าตัวเลขสัดส่วนระยะห่างขอบล่างของหน้าต่าง ตัวเลข 0.5 บ่งบอกว่าถ้าเลื่อนมาถึงกึ่งกลางชิ้นงานรายการสุดท้าย ให้เริ่มเรียกการโหลดข้อมูลหน้าถัดไปเตรียมสแตนด์บายได้ทันที

5.7 การจัดกลุ่มข้อมูลด้วย SectionList

เมื่อข้อมูลรายการมีความจำเป็นที่จะต้องจัดแยกจำแนกหมวดหมู่ (เช่น คลังรายชื่ออักษรขึ้นต้น A-Z, การแบ่งหมวดหมู่เมนูอาหาร) คอมโพเนนต์ `SectionList` (React Native, 2024) จะช่วยอำนวยความสะดวกในการจัดสรรความลื่นไหลได้เป็นหมวดหมู่:

โครงสร้างข้อมูลสำหรับ SectionList

ข้อมูลในพร็อพส์ `sections` ต้องเป็นอาร์เรย์ของออบเจกต์ ซึ่งมีคีย์ชื่อ `title` (หรือคีย์จัดกลุ่มใด ๆ) และคีย์อาร์เรย์ชื่อ `data` บรรจุรายละเอียดลูกย่อยในกลุ่มนั้น:

```
import { SectionList, Text, View, StyleSheet, SafeAreaView } from 'react-
native';

const MENU_DATA = [
  {
    title: 'ของคาว (Main Dishes)',
    data: ['ข้าวผัดปู', 'ต้มยำกุ้งน้ำข้น', 'ผัดไทยกุ้งสด'],
  },
  {
    title: 'เครื่องดื่ม (Drinks)',
    data: ['ชาไทยนมสด', 'กาแฟอเมริกาโน่เย็น'],
  },
];

export default function RestaurantMenu() {
  return (
    <SafeAreaView style={{ flex: 1 }}>
```

```
        <SectionList
          sections={MENU_DATA}
          keyExtractor={(item, index) => item + index}
      // ฟังก์ชันวาดเมนูอาหารย่อยแต่ละตัว
          renderItem={({ item }) => (
            <View style={styles.item}>
              <Text style={styles.itemText}>{item}</Text>
            </View>
      )}
      // ฟังก์ชันวาดป้ายแถบหัวข้อกลุ่ม (Sticky Header)
          renderSectionHeader={({ section: { title } }) => (
            <View style={styles.header}>
              <Text style={styles.headerText}>{title}</Text>
            </View>
      )}
     />
      </SafeAreaView>
  );
}

const styles = StyleSheet.create({
  header: {
    backgroundColor: '#3b82f6',
    padding: 10,
  },
  headerText: {
    fontSize: 16,
    fontWeight: 'bold',
    color: '#ffffff',
  },
  item: {
    padding: 16,
    borderBottomWidth: 1,
    borderBottomColor: '#e2e8f0',
  },
  itemText: {
    fontSize: 15,
  },
});
```

สรุปท้ายบทเรียน

ปัญหาประสิทธิภาพของ ScrollView กับข้อมูลขนาดใหญ่: ScrollView ทำงานแบบ Eager Rendering ซึ่งจะโหลดและวาดคอมโพเนนต์ย่อยทั้งหมดลงสู่หน่วยความจำ (RAM) พร้อมกันตั้งแต่เริ่มต้น จึงทำให้เมื่อต้องแสดงผลรายการข้อมูลจำนวนมหาศาลหรือมีรูปภาพประกอบจำนวน

มาก จะเกิดปัญหาหน่วยความจำล้นและทำให้อัตราการแสดงผลเฟรมเรตตกจนแอปพลิเคชันทำงานสะดุดหรือเด้งออก

การแก้ไขด้วย FlatList (Virtualization และ Recycling): เพื่อเพิ่มความลื่นไหลระดับเนทีฟ FlatList ถูกนำเข้ามาช่วยแก้ปัญหาโดยใช้แนวคิด Virtualization ที่เรนเดอร์เฉพาะข้อมูลที่ปรากฏบนขอบจอจริงเท่านั้น และใช้แนวคิด Item View Recycling ในการหมุนเวียนจัดสลับกล่องรายการที่เลื่อนหลุดขอบจอกลับมาสวมใส่ข้อมูลชุดถัดไป ช่วยประหยัดการสร้าง Native Views และประหยัดทรัพยากรระบบได้อย่างมีประสิทธิภาพ

โครงสร้างและพร็อพส์หลักของ FlatList: การประกาศใช้งาน FlatList ต้องการอ็อบชันควบคุมหลัก 3 ประการ ได้แก่ data สำหรับส่งผ่านอาร์เรย์ของข้อมูลดิบทั้งหมด, renderItem ซึ่งคอยทำหน้าที่สร้างรูปแบบ JSX ของแถบข้อมูลรายย่อย และ keyExtractor ที่ใช้กำหนดค่าสตริงคีย์เอกลักษณ์เฉพาะตัวให้แต่ละรายการเพื่อช่วยระบบ Diffing Engine ของ React ประเมินจุดเปลี่ยนได้อย่างรวดเร็ว

การดึงข้อมูลจาก REST API มาใช้ร่วมกับ FlatList: การประยุกต์ใช้งานในระบบจริงจะนำฟังก์ชัน fetch() มาประสานร่วมกับ useState และ useEffect เพื่อขอข้อมูลดิบผ่านระบบเครือข่ายอินเทอร์เน็ต โดยเราสามารถแสดงกล่อง ActivityIndicator บ่งบอกความก้าวหน้าขณะรอผลลัพธ์ ก่อนที่จะจัดเก็บข้อมูลลงในสเตตและนำข้อมูลดังกล่าวไปเป็นแหล่งอ้างอิงหลักในโครงสร้างของ FlatList

การปรับแต่งส่วนประกอบลิสต์: เพื่อป้องกันความซับซ้อนในการเขียนคอมโพเนนต์ซ้อนทับกัน FlatList ได้เตรียมเครื่องมือเฉพาะตัวมาอำนวยความสะดวก เช่น ListHeaderComponent และ ListFooterComponent สำหรับสร้างหัวและท้ายหน้าเลื่อน, ItemSeparatorComponent ในการวาดเส้นคั่นหรือจัดระเบียบช่องไฟระหว่างแถว และ ListEmptyComponent เพื่อแสดงข้อความแจ้งเตือนเมื่อไม่มีข้อมูลแสดงผล

ฟีเจอร์ Pull-to-Refresh และ Infinite Scroll: แอปพลิเคชันที่ทันสมัยสามารถยกระดับความสะดวกได้ด้วยระบบ Pull-to-Refresh ที่เปิดทางให้ผู้ใช้นิ้วปัดหน้าจอลงเพื่อรีเฟรชข้อมูลผ่านพร็อพส์ refreshing และ onRefresh รวมถึงการทำ Infinite Scroll เพื่อดาวน์โหลดหน้าถัดไปแบบต่อเนื่องผ่านพร็อพส์ onEndReached และการกำหนดสัดส่วนระยะเริ่มโหลดผ่าน onEndReachedThreshold

การจัดกลุ่มข้อมูลด้วย SectionList: สำหรับข้อมูลที่ต้องจัดแยกเป็นหมวดหมู่อย่างชัดเจน SectionList คือตัวเลือกในการแสดงผลที่ตอบโจทย์ โดยข้อมูลนำเข้าในพร็อพส์ sections จะต้องจัดเรียงกลุ่มข้อมูลในรูปแบบโครงสร้างออบเจกต์ที่มีคีย์ title สำหรับป้ายหัวข้อกลุ่ม และคีย์ data

สำหรับบรรจุรายการย่อย ซึ่งระบบจะทำการคำนวณและวาดป้ายแถบหัวข้อกลุ่ม (Sticky Header) ให้สอดคล้องกันโดยอัตโนมัติ

แบบฝึกหัดท้ายบทเรียน

แบบทดสอบนี้มีวัตถุประสงค์เพื่อประเมินความเข้าใจเกี่ยวกับปัญหาประสิทธิภาพเลื่อนไหล (Scrolling Performance) ข้อแตกต่างระหว่าง `ScrollView` และ `FlatList` โครงสร้างและพร็อพส์หลักในการควบคุมรายการ การเชื่อมต่อเน็ตเวิร์กดึงข้อมูลผ่าน API รวมถึงระบบ Pull-to-Refresh, Infinite Scroll และการจัดรูปเล่มประเภทกลุ่มด้วย `SectionList` ใน React Native Expo SDK 54

คำถามประจำบทเรียน (10 ข้อ)

ข้อ 1: ข้อใดคือสาเหตุหลักที่ทำให้อินเทอร์เฟซเกิดอาการหน่วง เฟรมเรตตก หรือแอปพลิเคชันค้างเด้งปิดตัว (Crash) เมื่อใช้งาน `ScrollView` แสดงผลข้อมูลจำนวนมาก?

ก. `ScrollView` ล็อกไม่ให้อุปกรณ์ดึงสัญญาณอินเทอร์เน็ต

ข. `ScrollView` ทำการเรนเดอร์และสร้างวัตถุลูกทั้งหมดลงสู่หน่วยความจำ (RAM) พร้อมกันในทันที แม้ว่าชิ้นส่วนข้อมูลจะยังไม่แสดงบนหน้าจอจริงก็ตาม

ค. `ScrollView` ห้ามเขียนจัดตำแหน่งบล็อกสีด้วยระบบ Flexbox

ง. `ScrollView` ไม่รองรับการแสดงผลกล่องอินพุต `TextInput`

ข้อ 2: กลไกแบบ Virtualization ที่ใช้งานภายในคอมโพเนนต์ `FlatList` ช่วยเพิ่มระดับประสิทธิภาพและประหยัดพื้นที่ RAM ได้อย่างไร?

ก. ย้ายเนื้อหางานคำนวณทั้งหมดส่งไปให้คอมพิวเตอร์หลังบ้านประมวลผลแทน

ข. แสดงผลเฉพาะชิ้นข้อมูลรายการที่กำลังอยู่ในขอบข่ายขนาดหน้าจอจริง และทำลาย/รียูสกล่องจัดแสดงของรายการที่หลุดหน้าจอออกไป

ค. ซ่อนข้อมูลทั้งหมดไว้ในเครื่องมือ AsyncStorage ตลอดเวลา

ง. ลดขนาดความละเอียดของภาพที่โหลดให้กลายเป็นภาพขาวดำอัตโนมัติ

ข้อ 3: พร็อพส์ (Props) ใดต่อไปนี้มีหน้าที่สำคัญในการเป็นตัวแปลงข้อมูลวัตถุอาร์เรย์ให้ออกมาเป็น JSX คอมโพเนนต์การ์ดแสดงผลบนหน้าจอลิสต์?

ก. `data`

ข. `renderItem`

ค. `keyExtractor`

ง. `ItemSeparatorComponent`

ข้อ 4: ในฟังก์ชันการดึงค่าของ `renderItem` ข้อใดระบุรูปแบบพารามิเตอร์แบบกระจายออบเจกต์ (Destructuring) ที่ถูกต้องสำหรับการสกัดข้อมูลไอเท็มปัจจุบันมาแสดง?

ก. `renderItem={(item) => <Text>{item.title}</Text>}`

ข. `renderItem={({ item }) => <Text>{item.title}</Text>}`

ค. `renderItem={(title) => <Text>{title}</Text>}`

ง. `renderItem={({ data }) => <Text>{data.title}</Text>}`

ข้อ 5: เหตุใดผู้พัฒนาแอปจึงสมควรใส่ใจกำหนดพร็อพส์ `keyExtractor` ให้ส่งค่าสตริงที่เป็นเอกลักษณ์ (Unique String Key) ให้แก่ FlatList ทุกครั้ง?

ก. เพื่อบังคับให้อุปกรณ์ระบุขนาดของรูปภาพในรายการโดยอัตโนมัติ

ข. เพื่อให้ React สามารถตรวจเช็คสถานะการเปลี่ยนแปลงตำแหน่ง ย้ายสลับ หรือลบรายการได้อย่างถูกต้องแบบเจาะจงเฉพาะชิ้นงาน โดยไม่ต้องเสียเวลาวาดใหม่ทั้งหมด

ค. เพื่อจำกัดไม่ให้ปุ่มสัมผัสทำงานได้เมื่อคีย์บอร์ดปรากฏขึ้น

ง. เพื่อระบุตัวเลขจำนวนหน้าสำหรับดาวน์โหลดข้อมูลเพิ่มเติม

ข้อ 6: เมื่อต้องการดึงรายการโพสต์หรือรูปภาพจาก RESTful API ทันทีที่หน้าจอลิสต์เริ่มต้นทำงาน (Mount) ใน React Native ข้อใดคือแนวทางปฏิบัติมาตรฐาน?

ก. เรียกใช้คำสั่ง `fetch()` ในสไตล์ชีทภายนอก

ข. เขียนฟังก์ชัน `fetch()` ครอบไว้ภายใน `useEffect` Hook ที่มีอาเรย์การพึ่งพาเป็นค่าว่าง (`[]`) เพื่อให้สัญญาณอินพุตทำงานเพียงรอบเดียว

ค. เรียกฟังก์ชันเน็ตเวิร์กภายใน Property ของ `keyExtractor`

ง. ใช้ `useState` ในการดักจับค่า IP ของโทรศัพท์แบบวนซ้ำ

ข้อ 7: พร็อพส์ใดของ `FlatList` ที่ทำหน้าที่แสดงคอมโพเนนต์อื่นทดแทน (เช่น ข้อความแจ้งว่าไม่มีรายการของ) เมื่อพบว่าไม่มีข้อมูลอยู่ในอาเรย์เลย?

ก. `ListFooterComponent`

ข. `ListHeaderComponent`

ค. `ListEmptyComponent`

ง. `ItemSeparatorComponent`

ข้อ 8: การเปิดความสามารถให้ผู้ใช้ลากขอบจอลงด้านล่างสุดเพื่อสั่งดึงข้อมูลชุดใหม่ (Pull-to-Refresh) ใน FlatList จะต้องทำรายการตั้งค่าพร็อพส์คู่ใดร่วมกัน?

ก. `refreshing` และ `onRefresh`

ข. `onEndReached` และ `onEndReachedThreshold`

ค. `active` และ `onActive`

ง. `secureTextEntry` และ `onChangeText`

ข้อ 9: พร็อพส์ `onEndReached` ในคอมโพเนนต์ `FlatList` มีพฤติกรรมการทำงานหลักอย่างไร?

ก. ทำการตรวจสอบสิทธิข้อมูลเมื่อผู้ใช้นิ้วเลื่อนออกจากปุ่มล้างค่า

ข. เรียกฟังก์ชันสั่งการดาวน์โหลดข้อมูลหน้าถัดไป (Infinite Scroll) ทันทีเมื่อผู้ใช้งานใช้นิ้วเลื่อนหน้าจอลิสต์ลงไปจนเกือบถึงส่วนท้ายสุดของตารางรายการ

ค. ล้างสเตตทั้งหมดกลับเป็นค่าเริ่มต้นเมื่อลิสต์แสดงผลเสร็จ

ง. เคลื่อนย้ายโครงสร้างหัวข้อของลิสต์ให้อยู่ชิดปุ่มล่างสุดของขอบอุปกรณ์

ข้อ 10: หากต้องการจัดเลย์เอาต์แสดงผลที่แยกข้อมูลออกเป็นหมวดหมู่ (เช่น คลังเครื่องดื่ม, คลังของหวาน) โดยต้องการให้มีหัวข้อกลุ่มลอยตัวค้างไว้เวลาเลื่อน (Sticky Headers) คอมโพเนนต์ใดตอบโจทย์งานลักษณะนี้ได้ดีที่สุด?

ก. `ScrollView`

ข. `FlatList`

ค. `SectionList`

ง. `SafeAreaView`

เฉลยคำตอบและคำอธิบายโดยละเอียด

เฉลย ข้อ 1: ตอบ ข.

• เหตุผลประกอบ:

• ก. ผิด: `ScrollView` ไม่ได้บล็อกการดึงเครือข่ายของระบบ

• ข. ถูก: `ScrollView` ไม่มีระบบหน่วยความจำแบบไดนามิก มันจะประมวลผลแปลง JSX ทุกชิ้นข้างใน (Eager Rendering) ไม่ว่าจะล้นจอไปยาวแค่ไหน ทำให้เปลือง RAM ทวีคูณตามสัดส่วนรายการข้อมูล

• ค. และ ง. ผิด: การเขียนจัดตำแหน่งและใส่อินพุตทำได้ตามรูปแบบทั่วไป

เฉลย ข้อ 2: ตอบ ข.

• เหตุผลประกอบ:

• ก. ผิด: งานเรนเดอร์ UI จะทำบนอุปกรณ์ปลายทางเสมอ ไม่ใช่หลังบ้าน

• ข. ถูก: `FlatList` ยึดหลักเรนเดอร์ขอบเขตเฉพาะส่วนที่มองเห็นจริง และทำการปล่อย RAM คืนระบบเมื่อวัตถุเลื่อนหลุดสายตา พร้อมทั้งดึงเฟรม View ที่ว่างเหล่านั้นกลับมารอรับสวมข้อมูลชิ้นถัดไป (Recycling) ส่งผลให้มีความคงทนสูงและประหยัด RAM

• **ค. และ ง. ผิด:** AsyncStorage มีไว้เก็บบันทึกถาวร และระบบไม่มีการสุ่มเปลี่ยนภาพเป็นขาวดำ

เฉลย ข้อ 3: ตอบ ข.

• **เหตุผลประกอบ:**

• **ก. ผิด:** `data` เป็นเพียงคลังอาร์เรย์ข้อเท็จจริง (ข้อมูลดิบ) ของโพสต์ แต่ยังไม่มีโครงสร้างสไตล์ภาพข้อความ

• **ข. ถูก:** พร็อพส์ `renderItem` ทำหน้าที่รับฟังก์ชันที่ผู้พัฒนาอธิบายสไตล์ไว้ เพื่อนำข้อมูลดิบในแต่ละดัชนี (Index) มาวาดแปลงให้กลายเป็นโครงสร้าง JSX เพื่อแสดงขึ้นจอจริง

• **ค. ผิด:** `keyExtractor` ทำหน้าที่กำหนดระบุข้อความกุญแจส่วนตัว ไม่ใช่ผู้วาด UI

• **ง. ผิด:** `ItemSeparatorComponent` ใช้สำหรับวาดเส้นแบ่งเส้นคั่นเท่านั้น

เฉลย ข้อ 4: ตอบ ข.

• **เหตุผลประกอบ:**

• **ก. ค. และ ง. ผิด:** การเขียนพารามิเตอร์โดยไม่มีปีกกาอาร์กิวเมนต์ `{}` หรือสับสนชื่อคีย์ จะนำเอาอ็อบเจกต์รวมของ MetaData (ซึ่งประกอบด้วยตัวชี้นงาน, ลำดับ Index, ข้อมูลภายนอก) มาใช้ตรง ๆ ส่งผลให้เกิดอาการค่าเป็น `undefined` หรือโปรแกรมล่ม

• **ข. ถูก:** พารามิเตอร์ที่ `renderItem` ส่งมาจะมีรูปเป็นออบเจกต์ `{ item, index, separators }` การใส่ปีกกาสกัดตัวแปรคีย์ชื่อ `item` (`{ item }`) จึงเป็นวิธีที่ถูกต้องดั้งเดิมในการเอาวัตถุเดี่ยวแต่ละดัชนีมาใช้งาน

เฉลย ข้อ 5: ตอบ ข.

• **เหตุผลประกอบ:**

• **ก. ผิด:** ขนาดของรูปภาพต้องถูกตั้งผ่าน CSS Style เท่านั้น

• **ข. ถูก:** ในสถาปัตยกรรม React การใช้คีย์เฉพาะช่วยประหยัดเวลาอย่างมาก เพราะถ้าเกิดการอัปเดตข้อมูลขึ้นมาจุดใดจุดหนึ่ง React จะดึงสตริงกุญแจตรวจสอบเพื่อประมวลผลขยับเฉพาะกล่อง View นั้นโดยไม่ต้องเรนเดอร์ใหม่หมดทั้งแผง ซึ่งเพิ่มความลื่นไหลเป็นอย่างมาก

• **ค. และ ง. ผิด:** ไม่ได้ควบคุมการทำงานปุ่ม และไม่ได้มีขอบข่ายเกี่ยวกับเลขหน้าข้อมูลดั้งเดิม

เฉลย ข้อ 6: ตอบ ข.

• **เหตุผลประกอบ:**

• **ก. ผิด:** สไตล์ชีทมีหน้าที่จัดสเปซเฉดสีเท่านั้น ห้ามเรียกนำข้อมูลเน็ตเวิร์ก

• **ข. ถูก:** การประยุกต์ใช้งาน `useEffect` พร้อมกำหนด Dependency Array ว่างเปล่า [] จะสั่งให้อุปกรณ์เชื่อมต่อทำงานส่งคำขอ Fetch API เพียงแค่ครั้งเดียวตอนโหลดหน้าจอ

ขึ้นมา (Component Did Mount) ซึ่งช่วยบันทึกข้อมูลสเตตและเตรียมนำไปใช้ป้อนต่อ FlatList ได้อย่างมีประสิทธิภาพ

- **ค. และ ง. ผิด:** คีย์เฉพาะและการทำสเตตไอพีไม่ได้เอื้อต่อการดึงค่า API โพสต์

เฉลย ข้อ 7: ตอบ ค.

- **เหตุผลประกอบ:**
  - **ก. และ ข. ผิด:** ใช้ตกแต่งส่วนบนและส่วนล่างสุดของลิสต์ตาราง
  - **ค. ถูก:** `ListEmptyComponent` คอยตรวจเช็คความยาวอาร์เรย์ใน `data` หากข้อมูลมีค่าว่าง จะเรนเดอร์คอมโพเนนต์อื่นที่เราส่งมอบเข้าไปแทน เช่น ภาพการ์ตูนกล่องเปล่าพร้อมคำเตือน "ไม่พบข้อมูลใด ๆ ในขณะนี้"
  - **ง. ผิด:** ใช้สำหรับคั่นระหว่างชิ้นงาน

เฉลย ข้อ 8: ตอบ ก.

- **เหตุผลประกอบ:**
  - **ก. ถูก:** เพื่ออำนวยฟังก์ชันกวาดดึงหน้าจอ (Pull-to-Refresh) เราต้องใช้ `refreshing` (สเตตคอยจำสถานะการดึงประมวลผล) ควบคู่กับเหตุการณ์ `onRefresh` (ฟังก์ชันสั่งยิงเรียกเน็ตเวิร์กอัปเดตใหม่)
  - **ข. ผิด:** เป็นคู่สำหรับระบุงานโหลดเพิ่มตอนท้ายจอ (Infinite Scroll)
  - **ค. และ ง. ผิด:** ไม่ใช่คำสั่งที่อยู่ในขอบข่ายงานดึงหมุนรีเฟรชหน้าต่าง

เฉลย ข้อ 9: ตอบ ข.

- **เหตุผลประกอบ:**
  - **ก. ค. และ ง. ผิด:** พฤติกรรมไม่ใช่การเคลียร์ปุ่ม ล้างสเตต หรือย้ายหัวข้อขยับขอบจอ
  - **ข. ถูก:** เหตุการณ์ `onEndReached` จะถูกส่งสัญญาณเมื่อผู้ใช้นิ้วเลื่อนตารางลงมาจนเข้าเกณฑ์ขอบล่าง เพื่อส่งสัญญาณทำ Callback ไปเรียกหน้าถัดไป (เช่น หน้า 2, 3) มาประกอบลงสเตตเดิมเพื่อสร้างลิสต์แบบต่อเนื่องยาวไม่มีที่สิ้นสุด

เฉลย ข้อ 10: ตอบ ค.

- **เหตุผลประกอบ:**
  - **ก. และ ข. ผิด:** ScrollView เปลือง RAM และไม่มี Sticky Headers ออโต้ ส่น FlatList ออกแบบมาสำหรับแถวเรียงเส้นตรง ไม่ได้สนับสนุนการแยกกลุ่มและป้าย Sticky ประจำเซกชันโดยตรง

• **ค. ถูก:** `SectionList` ดีไซน์ขึ้นมาแยกย่อยข้อมูลกลุ่มประเภทหัวข้อ (Sections) โดยความสามารถหลักคือมีระบบตรึงแถบหัวกลุ่มค้างไว้ (Sticky Section Headers) บนขอบจอยามเลื่อนผ่าน ซึ่งอำนวยความสวยงามสไตล์ระดับเนทีฟยอดนิยม

• **ง. ผิด:** มีหน้าที่ป้องกันเนื้อหาชนขอบบาก ไม่เกี่ยวกับการคำนวณลิสต์กลุ่ม

ปฏิบัติการท้ายบทเรียน: การพัฒนาแอปแสดงรายการรูปภาพจาก API พร้อมระบบรีเฟรชและโหลดข้อมูลเพิ่ม

ปฏิบัติการนี้จะฝึกทักษะการประยุกต์ใช้งานคอมโพเนนต์รายการประสิทธิภาพสูง `FlatList` ร่วมกับการดึงข้อมูลผ่านระบบเครือข่ายออนไลน์จริงด้วย Fetch API (ดึงข้อมูลจำลองจาก JSONPlaceholder) และอัปเดตข้อมูลแบบแบ่งหน้า (Pagination) เพื่อทำระบบโหลดอัตโนมัติเมื่อเลื่อนขอบล่างสุด (Infinite Scroll) พร้อมระบบดึงลงรีเฟรชหน้าจอ (Pull-to-Refresh)

1. วัตถุประสงค์การเรียนรู้

1. สามารถดึงข้อมูลภาพและข้อความจาก REST API ออนไลน์ภายนอกผ่าน Fetch API ร่วมกับ `useEffect` ได้สำเร็จ

2. สามารถจัดสรรหน่วยความจำและเรนเดอร์ภาพจากอินเทอร์เน็ตปริมาณมากในระบบรายการ `FlatList` ได้อย่างลื่นไหล

3. สามารถพัฒนาฟังก์ชัน Pull-to-Refresh เพื่อเคลียร์สเตตกลับคืนหน้า 1 และประมวลผลข้อมูลใหม่จากเซิร์ฟเวอร์

4. สามารถทำระบบแบ่งหน้าโหลดเนื้อหาต่อด้านท้าย (Infinite Scroll) ด้วยพร็อพส์ `onEndReached`

5. สามารถเขียนจัดวางและตกแต่งชิ้นส่วนย่อยประกอบลิสต์ (`ListHeaderComponent`, `ListFooterComponent`, `ListEmptyComponent`, และ `ItemSeparatorComponent`) ได้ถูกต้อง

2. โจทย์และข้อกำหนดปฏิบัติการ (Requirements)

ผู้เรียนจะต้องเขียนพัฒนาโมเดลจำลอง **แอปพลิเคชันคลังแกลเลอรีภาพออนไลน์ (Live Photo Gallery Stream)** โดยประกอบด้วยเงื่อนไข 2 ส่วนหลักดังนี้:

1. ชิ้นส่วนการ์ดภาพย่อย: `components/PhotoCard.tsx`

• สร้างเป็น Custom Component เพื่อแสดงชิ้นผลงานแต่ละชิ้นประกอบด้วย:

• รูปภาพขนาดย่อ (`thumbnailUrl`) แสดงผลทรงสี่เหลี่ยมโค้งมนสวยงาม

• ข้อความหัวเรื่องรูปภาพ (`title`) บรรยายรายละเอียด

• ตัวเลขดัชนีระบุ ID ของรูปภาพ

• จัดความปลอดภัยด้วย TypeScript Interface `PhotoCardProps`

• ตัวปุ่มการ์ดจัดด้วย `Pressable` ที่มีลูกเล่นการกระพริบสว่างหรือยืดหดน้ำหนักสัมผัสเมื่อถูกคลิก

2. หน้าหลักควบคุมแบบต่อเนื่อง: `app/index.tsx`

• ดึงข้อมูลผ่าน URL: `https://jsonplaceholder.typicode.com/photos`

• **การแบ่งหน้าโหลดข้อมูล (Pagination):** กำหนดให้โหลดข้อมูลเพียงครั้งละ 10 รูป (`_limit=10`) โดยส่งผ่านพารามิเตอร์ `_page` เพื่อระบุหน้าข้อมูล (เช่น หน้า 1 โหลดรูป 1-10, หน้า 2 โหลดรูป 11-20)

• **การดึงเพื่อรีเฟรช (Pull-to-Refresh):** เมื่อดึงหน้าจอลง จะเรียกโหลดข้อมูลหน้า 1 ใหม่ พร้อมสั่งล้างคลังสเตตอาร์เรย์เดิมทิ้ง

• **การเลื่อนลงสุดเพื่อโหลดต่อ (Infinite Scroll):** เมื่อใช้นิ้วเลื่อนลงมาจนถึงระยะกึ่งกลางของไอเท็มสุดท้าย จะทริกเกอร์เรียกโหลดหน้าถัดไป (`page + 1`) เพื่อนำรูปภาพชุดใหม่ไปประกอบต่อท้ายอาร์เรย์เดิม พร้อมเรนเดอร์ไอคอนหมุนโหลดข้อมูล (`ActivityIndicator`) ค้างไว้ที่ด้านล่างสุดของตารางรายการ

• **องค์ประกอบตกแต่ง:**

• **Header:** แสดงข้อความใหญ่ "Photo Stream" และบอกยอดรูปสะสมปัจจุบัน

• **Separator:** เส้นคั่นสีเทาบางกั้นระหว่างภาพ

• **Empty:** โชว์ข้อความ "ไม่พบข้อมูลรูปภาพในคลัง" กรณีสเตตอาร์เรย์เป็นศูนย์

3. ขั้นตอนการลงมือปฏิบัติ (Step-by-Step Tutorial)

ขั้นตอนที่ 1: การสร้าง Custom Component สำหรับจัดสไตล์ชิ้นการ์ดรูปภาพ

สร้างไฟล์คอมโพเนนต์สำหรับการ์ดแสดงรายการรูปภาพตามเส้นทาง `components/PhotoCard.tsx` และใส่โครงสร้างโค้ดดังนี้:

```
import { View, Text, Image, StyleSheet, Pressable } from 'react-native';

// กำหนดประเภทข้อมูล Props ด้วย TypeScript Interface
export interface PhotoCardProps {
  id: number;
  title: string;
  thumbnailUrl: string;
  onPressCard?: () => void;
}
```

```
export default function PhotoCard({ id, title, thumbnailUrl, onPressCard
}: PhotoCardProps) {
  return (
    <Pressable
      onPress={onPressCard}
      style={({ pressed }) => [
        styles.card,
        { backgroundColor: pressed ? '#1e293b' : '#0f172a' } // เปลี่ยนสีพื้นหลัง
เล็กน้อยเมื่อแตะ
   ]}
    >
      {/* แสดงรูปภาพจาก URI เครือข่าย */}
      <Image source={{ uri: thumbnailUrl }} style={styles.thumbnail} />

      <View style={styles.textContainer}>
        <Text style={styles.idText}>PHOTO ID: #{id}</Text>
        <Text style={styles.titleText} numberOfLines={2}>
          {title}
        </Text>
      </View>
    </Pressable>
 );
}

const styles = StyleSheet.create({
  card: {
    flexDirection: 'row',
    padding: 12,
    alignItems: 'center',
    borderRadius: 8,
  },
  thumbnail: {
    width: 60,
    height: 60,
    borderRadius: 8,
    backgroundColor: '#334155', // พื้นหลังสีเทาขณะรอดึงรูปจริง
  },
  textContainer: {
    flex: 1,
    marginLeft: 16,
  },
  idText: {
    fontSize: 11,
    color: '#38bdf8',
    fontWeight: 'bold',
    marginBottom: 2,
  },
  titleText: {
    fontSize: 14,
```

```
    fontWeight: '500',
    color: '#cbd5e1',
    textTransform: 'capitalize',
  },
});
```

ขั้นตอนที่ 2: วางหน้าจอเขียนฟังก์ชัน Fetch API แบบแบ่งหน้าในไฟล์หลัก app/index.tsx

เขียนประมวลสเตตจัดการหน้าข้อมูลสัญญานเน็ตเวิร์ก พร้อมประกอบคำสั่งลงในหน้าหลัก:

```
import { useState, useEffect } from 'react';
import {
  StyleSheet,
  Text,
  View,
  FlatList,
  ActivityIndicator,
  SafeAreaView,
  Alert
} from 'react-native';
import PhotoCard from '../components/PhotoCard'; // โหลดการ์ดภาพย่อย

interface Photo {
  id: number;
  title: string;
  thumbnailUrl: string;
  url: string;
}

export default function PhotoStreamScreen() {
 // สเตตเก็บคลังอาร์เรย์รูปภาพ
  const [photos, setPhotos] = useState<Photo[]>([]);

 // สเตตจัดการคีย์เลขหน้าและการจำสถานะหมุนโหลด
  const [page, setPage] = useState<number>(1);
  const [loadingInitial, setLoadingInitial] = useState<boolean>(true); //
โหลดครั้งแรกสุด
  const [loadingMore, setLoadingMore] = useState<boolean>(false);        //
โหลดเพิ่มส่วนท้าย
  const [refreshing, setRefreshing] = useState<boolean>(false);          //
ดึงหน้าจอลงรีเฟรช
  const [hasMoreData, setHasMoreData] = useState<boolean>(true);         //
เช็คว่าหมดคลังหรือยัง

  const LIMIT = 10; // ดึงข้อมูลครั้งละ 10 รายการ

 // 1. ฟังก์ชันดึงข้อมูลผ่านเครือข่าย
  const fetchPhotosData = async (targetPage: number, isRefreshing: boolean
```

```
= false) => {
    try {
      if (isRefreshing) setRefreshing(true);
      else if (targetPage > 1) setLoadingMore(true);

      const url =
`https://jsonplaceholder.typicode.com/photos?_page=${targetPage}&_limit=${L
IMIT}`;
      const response = await fetch(url);
      const data: Photo[] = await response.json();

      if (data.length < LIMIT) {
        setHasMoreData(false); // หากข้อมูลส่งมาไม่ครบตามจำนวนจำกัด แปลว่าสิ้นสุดคลังแล้ว
      }

      if (isRefreshing) {
        setPhotos(data); // ล้างสเตตเดิมและใส่ข้อมูลหน้า 1 ชุดใหม่ลงไปตรงๆ
        setHasMoreData(true);
      } else {
        setPhotos((prev) => [...prev, ...data]); // นำไปต่อยอดพรีเพนท์ข้อมูลเดิม
      }
    } catch (error) {
      console.error('การดึงข้อมูลเครือข่ายผิดพลาด:', error);
      Alert.alert('เกิดข้อผิดพลาด', 'ไม่สามารถเชื่อมต่อสัญญาณอินเทอร์เน็ตได้ในขณะนี้');
    } finally {
      setLoadingInitial(false);
      setLoadingMore(false);
      setRefreshing(false);
    }
  };

 // ดึงข้อมูลหน้า 1 ตอนรันหน้าแอปเริ่มต้น
  useEffect(() => {
    fetchPhotosData(1);
  }, []);

 // 2. ฟังก์ชันสั่งลากนิ้วรีเฟรชด้านบน
  const handleRefresh = () => {
    setPage(1);
    fetchPhotosData(1, true);
  };

 // 3. ฟังก์ชันโหลดเพิ่มต่อท้ายเมื่อเลื่อนลงสุดหน้าจอ
  const handleLoadMore = () => {
    if (!loadingMore && hasMoreData) {
      const nextPage = page + 1;
      setPage(nextPage);
      fetchPhotosData(nextPage);
    }
```

```
  };

  // ... รายละเอียดสปอตไลท์ Render จะแยกอยู่ในส่วนเฉลยท้ายสุด
  return null;
}
```

4. โค้ดเฉลยฉบับสมบูรณ์ (Complete Solution)

ผู้เรียนสามารถคัดลอกรายละเอียดโค้ดโปรเจคฉบับสมบูรณ์ที่จัดวางสไตล์ชีทแบบสเปซโมเดิร์นสีเข้ม (Dark Mode) ไปรันผลลัพธ์ผ่านตัวจำลองเพื่อทดสอบระบบดึงภาพได้ทันที:

4.1 ซอร์สโค้ดไฟล์ย่อย: components/PhotoCard.tsx

```
import { View, Text, Image, StyleSheet, Pressable } from 'react-native';

export interface PhotoCardProps {
  id: number;
  title: string;
  thumbnailUrl: string;
  onPressCard?: () => void;
}

export default function PhotoCard({ id, title, thumbnailUrl, onPressCard
}: PhotoCardProps) {
  return (
    <Pressable
      onPress={onPressCard}
      style={({ pressed }) => [
        styles.card,
        { backgroundColor: pressed ? '#1e293b' : '#1e293b' }
   ]}
    >
      <Image source={{ uri: thumbnailUrl }} style={styles.thumbnail} />

      <View style={styles.textContainer}>
        <Text style={styles.idText}>ID: #{id}</Text>
        <Text style={styles.titleText} numberOfLines={2}>
          {title}
        </Text>
      </View>
    </Pressable>
 );
}

const styles = StyleSheet.create({
  card: {
    flexDirection: 'row',
    padding: 14,
    alignItems: 'center',
```

```
    borderRadius: 12,
    borderWidth: 1,
    borderColor: '#334155',
  },
  thumbnail: {
    width: 64,
    height: 64,
    borderRadius: 8,
    backgroundColor: '#334155',
  },
  textContainer: {
    flex: 1,
    marginLeft: 16,
  },
  idText: {
    fontSize: 10,
    color: '#38bdf8',
    fontWeight: 'bold',
    marginBottom: 2,
    letterSpacing: 0.5,
  },
  titleText: {
    fontSize: 14,
    fontWeight: '500',
    color: '#cbd5e1',
    textTransform: 'capitalize',
    lineHeight: 18,
  },
});
```

4.2 ซอร์สโค้ดหน้าจอหลักเชื่อม API: **app/index.tsx**

```
import { useState, useEffect } from 'react';
import {
  StyleSheet,
  Text,
  View,
  FlatList,
  ActivityIndicator,
  SafeAreaView,
  Alert
} from 'react-native';
import PhotoCard from '../components/PhotoCard';

interface Photo {
  id: number;
  title: string;
  thumbnailUrl: string;
```

```
  url: string;
}

export default function App() {
  const [photos, setPhotos] = useState<Photo[]>([]);
  const [page, setPage] = useState<number>(1);
  const [loadingInitial, setLoadingInitial] = useState<boolean>(true);
  const [loadingMore, setLoadingMore] = useState<boolean>(false);
  const [refreshing, setRefreshing] = useState<boolean>(false);
  const [hasMoreData, setHasMoreData] = useState<boolean>(true);

  const LIMIT = 10;

  const fetchPhotosData = async (targetPage: number, isRefreshing: boolean
= false) => {
    try {
      if (isRefreshing) setRefreshing(true);
      else if (targetPage > 1) setLoadingMore(true);

      const url =
`https://jsonplaceholder.typicode.com/photos?_page=${targetPage}&_limit=${
LIMIT}`;
      const response = await fetch(url);
      const data: Photo[] = await response.json();

      if (data.length < LIMIT) {
        setHasMoreData(false);
      }

      if (isRefreshing) {
        setPhotos(data);
        setHasMoreData(true);
      } else {
        setPhotos((prev) => [...prev, ...data]);
      }
    } catch (error) {
      console.error('การดึงข้อมูลเครือข่ายผิดพลาด:', error);
      Alert.alert('เกิดข้อผิดพลาด', 'ไม่สามารถเชื่อมต่อสัญญาณอินเทอร์เน็ตได้ในขณะนี้');
    } finally {
      setLoadingInitial(false);
      setLoadingMore(false);
      setRefreshing(false);
    }
  };

  useEffect(() => {
    fetchPhotosData(1);
  }, []);

  const handleRefresh = () => {
```

```
    setPage(1);
    fetchPhotosData(1, true);
  };

  const handleLoadMore = () => {
    if (!loadingMore && hasMoreData && !loadingInitial) {
      const nextPage = page + 1;
      setPage(nextPage);
      fetchPhotosData(nextPage);
    }
  };

// A. ส่วนตกแต่งแถบหัวของรายการ
  const renderHeader = () => (
   <View style={styles.header}>
      <Text style={styles.headerTitle}>Photo Gallery Feed</Text>
      <Text style={styles.headerSubtitle}>
    จำนวนภาพแสดงในสเตริม: {photos.length} ภาพ
      </Text>
    </View>
);

// B. ตัวแสดงแถบหมุนประมวลผลเพิ่มใต้ตารางลิสต์
  const renderFooter = () => {
    if (!loadingMore) return null;
    return (
      <View style={styles.footerLoader}>
        <ActivityIndicator size="small" color="#38bdf8" />
        <Text style={styles.footerLoaderText}>กำลังโหลดรูปภาพเพิ่มเติม...</Text>
      </View>
  );
  };

// C. ป้ายเตือนในกรณีลิสต์มีค่าว่าง
  const renderEmpty = () => (
    <View style={styles.emptyContainer}>
      <Text style={styles.emptyText}>ไม่พบข้อมูลรูปภาพในคลังจำลอง</Text>
    </View>
);

// D. เส้นประคั่นกลางแบ่งรายการ
  const renderSeparator = () => <View style={styles.separator} />;

  if (loadingInitial) {
    return (
      <View style={styles.centerContainer}>
        <ActivityIndicator size="large" color="#38bdf8" />
        <Text style={styles.loadingText}>กำลังเปิดแกลเลอรีภาพออนไลน์...</Text>
```

```
      </View>
  );
  }

  return (
    <SafeAreaView style={styles.safeArea}>
      <View style={styles.container}>
        <FlatList
          data={photos}
          keyExtractor={(item) => item.id.toString()}
          renderItem={({ item }) => (
            <PhotoCard
              id={item.id}
              title={item.title}
              thumbnailUrl={item.thumbnailUrl}
              onPressCard={() => Alert.alert(`ภาพรหัส #${item.id}`,
item.title)}
      />
     )}
     // ผูกสไตล์องค์ประกอบตกแต่งย่อย
          ListHeaderComponent={renderHeader}
          ListFooterComponent={renderFooter}
          ListEmptyComponent={renderEmpty}
          ItemSeparatorComponent={renderSeparator}

     // ผูกความสามารถรีเฟรชด้วยนิ้วลาก
          refreshing={refreshing}
          onRefresh={handleRefresh}

     // ผูกความสามารถ Infinite Scroll
          onEndReached={handleLoadMore}
          onEndReachedThreshold={0.5} // เลื่อนมาถึงขอบล่าง 50% ของรายการสุดท้ายให้ดึงหน้าใหม่
รอทันที

          contentContainerStyle={styles.listContent}
    />
      </View>
    </SafeAreaView>
 );
}

const styles = StyleSheet.create({
  safeArea: {
    flex: 1,
   backgroundColor: '#0f172a',
  },
  container: {
    flex: 1,
  },
```

```
  centerContainer: {
    flex: 1,
    backgroundColor: '#0f172a',
    justifyContent: 'center',
    alignItems: 'center',
  },
  loadingText: {
    color: '#94a3b8',
    marginTop: 12,
   fontSize: 14,
  },
  listContent: {
    paddingHorizontal: 16,
    paddingBottom: 24,
  },
  header: {
    paddingVertical: 20,
    borderBottomWidth: 1,
    borderBottomColor: '#1e293b',
    marginBottom: 16,
  },
  headerTitle: {
    fontSize: 24,
   fontWeight: 'bold',
    color: '#ffffff',
  },
  headerSubtitle: {
    fontSize: 12,
    color: '#94a3b8',
    marginTop: 4,
  },
  separator: {
    height: 12, // ทำหน้าที่เป็นระยะช่องไฟคั่นระหว่างไอเท็มแต่ละตัว
  },
  footerLoader: {
    paddingVertical: 20,
    flexDirection: 'row',
    justifyContent: 'center',
    alignItems: 'center',
    gap: 8,
  },
  footerLoaderText: {
    color: '#94a3b8',
    fontSize: 12,
  },
  emptyContainer: {
    padding: 40,
    alignItems: 'center',
```

```
    justifyContent: 'center',
  },
  emptyText: {
    color: '#64748b',
    fontSize: 16,
  },
});
```

เอกสารอ้างอิง

JSONPlaceholder. (2024). *JSONPlaceholder - Free Fake REST API*. จาก https://jsonplaceholder.typicode.com/

React Native. (2024). *FlatList API Reference*. จาก https://reactnative.dev/docs/flatlist

React Native. (2024). *SectionList API Reference*. จาก https://reactnative.dev/docs/sectionlist